RepoPilot

PRODUCT REQUIREMENTS DOCUMENT

AI-Powered Open-Source Engineering Agent

From GitHub issue to verified, review-ready pull request.

Document control

Property	Decision
Version	1.0
Status	Approved for implementation planning
Product category	AI Developer Tools / Open Source Automation
Target release	Hacktoberfest 2026 Hackathon MVP
Target users	Open-source contributors, maintainers and student developers
AI model	Gemma 4 API
Application stack	React, TypeScript, Node.js, Express
Database	MongoDB
Execution	Docker sandbox
Cloud	DigitalOcean
Repository scope	Public GitHub repositories
Development model	Solo developer; 24-hour target sprint

Product summary

RepoPilot connects to a public GitHub repository, understands a reported software problem, uses Gemma 4 to investigate relevant code, prepares a possible fix, tests it in an isolated Docker environment and—after a human approves it—can prepare a pull request.

This document defines the product scope, user needs, features, architecture, technology choices, security requirements, APIs, acceptance criteria and hackathon delivery priorities. It is the product-level source of truth; implementation-specific contracts can be detailed in a separate Technical Design Document.

1. Executive Summary

1.1 Product Overview

RepoPilot is an AI-powered open-source engineering agent that helps developers understand GitHub repositories, investigate software issues, generate code patches, validate proposed fixes and prepare pull requests.

Unlike a conventional AI chatbot, RepoPilot is organized around an end-to-end engineering workflow:

GitHub Issue → Repository Investigation → Root-Cause Analysis → Code Patch → Automated Testing → Human Approval → Pull Request

1.2 Vision and Mission

Vision: Make meaningful open-source contributions accessible to every developer, regardless of experience with unfamiliar codebases.

Mission: Reduce the time and technical complexity involved in understanding, investigating, fixing and contributing to open-source software.

1.3 Product Promise

RepoPilot helps developers move from identifying a software problem to preparing a tested, reviewable contribution. It does not autonomously merge code or guarantee that generated fixes are correct. Human approval remains mandatory before publishing changes.

2. Problem Statement

Open-source development relies on contributors and maintainers who spend significant time investigating and resolving software issues.

- Understanding unfamiliar repositories and project conventions.
- Identifying relevant files, functions and dependencies.
- Investigating incomplete or complicated bug reports.
- Reproducing software problems and identifying root causes.
- Generating focused code fixes and appropriate tests.
- Preparing clear pull requests and reviewing contributions.
- Helping beginners make their first meaningful contribution.

2.1 Target Problem

A developer discovers a GitHub issue but does not know which files are responsible, what caused the problem, how to reproduce it, how to implement a fix, whether the fix works or how to submit a contribution. RepoPilot assists with these steps in one controlled workflow.

3. Target Users and Personas

Persona	Problems	Needs
Beginner contributor	GitHub workflow is unfamiliar; repositories feel intimidating; unsure which issue or file to start with.	Simple explanations, suitable issues, relevant-file guidance and step-by-step contribution help.
Experienced developer	Time spent understanding unfamiliar code, reproducing bugs and repeating debugging tasks.	Fast repository analysis, useful patches, reproducible tests and clear review evidence.
Open-source maintainer	High issue volume, duplicate reports and limited time to investigate low-quality contributions.	Issue classification, duplicate candidates, faster investigation and review-ready proposals.

4. Product Goals and Success Metrics

4.1 Primary Goals

- Connect public GitHub repositories and retrieve their metadata and issues.
- Understand enough repository context to investigate a selected issue.
- Analyze and classify issues using Gemma 4.
- Identify relevant source files and likely causes.
- Generate proposed code patches and validate them.
- Execute tests in isolated Docker containers.
- Generate an evidence-based report.
- Create a pull request after explicit user approval.

4.2 MVP Success Metrics

Metric	MVP success target
Repository connection	Valid public repository metadata is retrieved.
Issue retrieval	Actual open GitHub issues are displayed.
AI investigation	Structured analysis is returned and validated.
File identification	Relevant source files are displayed with paths.
Patch generation	A valid patch is generated for a supported task.
Patch application	Patch applies to the temporary workspace.
Test execution	Actual Docker test result and exit status are captured.
Evidence report	Available investigation and execution evidence is shown.
PR preparation	Authorized pull request is created after approval.
Human approval	No publishing occurs without explicit approval.

These are acceptance targets, not claims of achieved performance. A passing test means the selected tests passed in the configured environment; it does not prove that all defects are absent.

5. Product Scope

5.1 P0 — Must-Have MVP

- Public GitHub repository integration.
- Repository overview and structure retrieval.
- Open issue retrieval and selection.
- AI issue analysis and classification.
- Duplicate issue candidate detection.
- Relevant file discovery and root-cause investigation.
- AI-generated code patches.
- Patch validation and application.
- Docker-based test execution.
- Patch Evidence Report.
- Human-approved pull request creation.

5.2 P1 — Secondary Features

- Screenshot-to-bug analysis, if the selected Gemma endpoint supports images.
- Beginner-friendly issue recommendations.
- Repository Knowledge Map.
- Difficulty estimation.

- Agent execution timeline.

5.3 Out of Scope

- Private repositories and organization-wide access.
- Automatic pull-request merging.
- Blockchain functionality.
- Multiple independent autonomous agents.
- Custom model training or fine-tuning.
- Full browser-based IDE.
- Enterprise team management.
- Every programming language and runtime.
- Advanced organization analytics.

6. Functional Requirements

FR-01 — Authentication

Users authenticate through GitHub OAuth. Requirements: secure server-side token handling, session management, logout, user profile retrieval and minimum required permissions. Acceptance: user can authenticate; account identity is known; credentials are never exposed to the browser or AI model.

FR-02 — Repository Connection

Users submit a public GitHub repository URL. Validate it and retrieve owner, name, default branch, primary language, description, README and repository tree. Acceptance: valid public repositories connect; invalid or inaccessible repositories produce clear errors.

FR-03 — Issue Discovery

Retrieve and display actual open GitHub issues with title, description, number, labels and creation date. Support pagination and exclude pull requests from the issue list. Acceptance: selecting an issue opens its details; API failures are visible.

FR-04 — AI Issue Classification

Gemma 4 classifies issues as bug, feature request, documentation, enhancement, question or other. Return summary, expected and actual behaviour, missing information, severity estimate and complexity estimate. Acceptance: structured output is validated; estimates are labelled as estimates.

FR-05 — Duplicate Issue Detection

Compare issue titles and descriptions; use semantic similarity where available; display related candidates and explain why they may be related. Acceptance: candidates are presented as suggestions; issues are never automatically closed.

FR-06 — Repository Intelligence

Read README, retrieve file tree, identify languages and package manifests, inspect dependencies, search filenames and symbols, and retrieve selected source files. Use GitHub API, bounded keyword/symbol search and Tree-sitter where practical. Acceptance: inspected files are shown and retrieval respects size/context limits.

FR-07 — Root-Cause Investigation

Provide the issue and relevant source context to Gemma 4. Return possible causes, supporting evidence, files that may need changes, uncertainty and missing information. Acceptance: structured findings reference available source context and do not claim certainty without evidence.

FR-08 — Code Patch Generation

Generate unified diffs or structured file changes. Validate format and paths, reject path traversal, prevent writes outside the workspace and show modifications for review. Acceptance: valid patches apply to the temporary copy; invalid patches are rejected.

FR-09 — Docker Sandbox Execution

Test proposed changes in an isolated environment. Initially support Node.js projects. Create a temporary workspace, apply the patch, use predefined runner images, execute approved commands, capture stdout/stderr, exit code and duration, enforce resource limits and clean up. Acceptance: actual commands run; failed and timed-out tests are reported honestly.

FR-10 — Agent Execution Timeline

Display real timestamped stages such as issue analysis, file retrieval, patch generation and test completion. Use Server-Sent Events. Acceptance: events originate from backend execution; failures are clearly marked; progress is not simulated.

FR-11 — Patch Evidence Report

Include repository and issue, investigation summary, files inspected and changed, diff, test commands, output, exit status, duration and PR URL if available. Acceptance: every completed run has a report; untested changes are explicitly labelled.

FR-12 — Pull Request Creation

After patch review and explicit approval, create or retrieve the user's fork, create a branch, commit the patch and open a PR against the original repository. Acceptance: correct base branch; no direct write to original repository; PR URL returned; no automatic merge.

FR-13 — Beginner Contribution Mode

Allow language and experience selection; recommend suitable issues; explain them simply; identify files and provide learning steps. Acceptance: explanations are understandable and difficulty is labelled as an estimate. This feature is deferred if core MVP delivery is at risk.

FR-14 — Screenshot-to-Bug Analysis

Allow validated image upload and multimodal Gemma analysis where supported. Link findings to repository investigation. Acceptance: use only a verified image-capable endpoint; a screenshot is not treated as proof of execution.

7. Technology Stack

7.1 Frontend

Technology	Responsibility
React + TypeScript	Component-based user interface and type safety.
Vite	Development server and build tooling.
Tailwind CSS	Responsive styling and design system.
React Router	Application navigation.
Redux Toolkit	Shared client-side state where needed.
Axios	HTTP communication with backend.
Lucide React	Icons.
Monaco Editor	Optional code and diff review interface.
React Markdown	Render AI explanations and reports.

7.2 Backend

Technology	Responsibility
Node.js + Express	Backend runtime and HTTP API.
TypeScript	Type safety and maintainability.
Zod	Request and response validation.
Octokit	GitHub API integration.
Supported Gemma API SDK	AI inference and model requests.
Mongoose	MongoDB object modelling.
SSE	Live execution updates.
Pino	Structured application logging.
BullMQ + Redis	Optional background job processing.

7.3 AI Layer

Selected model: Gemma 4 through a supported API. Responsibilities include issue understanding, classification, duplicate analysis, code reasoning, relevant-file selection, root-cause investigation, patch generation, PR descriptions and screenshot analysis where supported.

- Structured JSON outputs and schema validation.
- Prompt versioning and bounded context.
- Timeout and retry policies.
- Token and request usage tracking.
- Tool permission validation.
- No unrestricted shell execution.

7.4 Database

Selected database: MongoDB. Initial collections:

Collection	Important fields
users	_id, githubId, username, avatarUrl, encryptedCredentialReference, createdAt, updatedAt
repositories	_id, owner, name, fullName, githubRepositoryId, defaultBranch, primaryLanguage, description, readmeSummary, indexingStatus
agentRuns	_id, userId, repositoryId, issueNumber, status, currentStage, investigation, relevantFiles, generatedPatch, testResults, error, startedAt, completedAt
reports	_id, runId, repository, issue, investigationSummary, filesChanged, patch, testEvidence, pullRequestUrl, createdAt

Never store raw GitHub tokens. Encrypt credentials at rest, keep secrets out of prompts and apply user-level authorization to every resource.

7.5 Infrastructure

Technology	Responsibility
DigitalOcean App Platform	Host web application and backend where suitable.
DigitalOcean Spaces	Optional report and artifact storage.
Docker	Isolated test execution.
MongoDB Atlas or managed MongoDB	Database hosting.
Redis	Optional queue and job management.
GitHub API	Repository integration.
Gemma 4 API	AI inference.

The Docker worker may need a separate compute host if the selected application hosting environment does not support safe container execution. Verify service capabilities before deployment.

8. High-Level Architecture

The system consists of a React frontend, Express backend, GitHub integration service, Gemma integration service, repository retrieval service, controlled agent orchestrator, Docker execution worker, MongoDB database and DigitalOcean infrastructure.

8.1 Controlled Agent Stages

Stage	Responsibility
Issue Analyzer	Understand and classify the issue; extract expected behaviour.
Repository Investigator	Retrieve repository structure and relevant source files.
Root-Cause Analyzer	Propose likely causes with supporting evidence and uncertainty.
Patch Generator	Generate and validate a code diff; apply it to a temporary workspace.
Test Validator	Run approved tests in Docker and capture results.
PR Preparation	Prepare title and description, request approval and create PR.

These are stages coordinated by one backend-controlled orchestrator. They do not need to be separate autonomous agents. Gemma may request a tool action, but the backend validates permissions and executes it.

9. API Requirements

Base path: /api. All endpoints must validate input, authenticate and authorize protected resources, return consistent errors, log relevant events and apply rate limits where appropriate.

Method	Endpoint	Purpose
POST	/auth/github	GitHub authentication
POST	/repositories/connect	Connect repository
GET	/repositories/:id	Repository details
GET	/repositories/:id/issues	Retrieve issues
POST	/agent/analyze	Start agent run
GET	/agent/:runId	Retrieve agent run
GET	/agent/:runId/events	Stream execution events
POST	/agent/:runId/test	Start approved test run

Method	Endpoint	Purpose
POST	/agent/:runId/approve	Approve publishing
POST	/agent/:runId/pull-request	Create pull request
GET	/reports/:runId	Retrieve evidence report

10. Non-Functional Requirements

Security

- Secure authentication and session handling.
- Server-side credential protection and least-privilege GitHub permissions.
- Input validation, rate limiting and audit logging.
- Treat repository content and issue descriptions as untrusted input.
- Validate every model-requested tool action.
- Sandbox isolation and path traversal protection.
- No production secrets in test containers.

Reliability

- Graceful API failure handling.
- Timeouts for model requests.
- Retry only safe operations.
- Idempotency for publishing operations.
- Persistent agent run status.
- Clear recovery from failed stages.

Performance

- Paginate GitHub results.
- Limit repository context retrieval.
- Use background jobs for long-running work where needed.
- Stream progress updates.
- Avoid unnecessary repeated model requests.

Maintainability

- Modular service architecture.
- TypeScript types and Zod validation.
- Consistent error handling and structured logs.
- Environment-based configuration.
- Unit and integration tests.

Usability

- Beginner-friendly explanations.
- Visible agent progress.
- Clear patch review.
- Understandable test output.
- No hidden publishing actions.

11. User Experience Requirements

Screen	Required content
Dashboard	Branding, repository URL input, connect action, recent repositories and recent runs.
Repository Overview	Name, owner, description, language, README summary, file structure and open issues.
Issue Explorer	Issue title, description, labels, number, related issues, classification and investigate action.
Agent Workspace	Current stage, activity timeline, investigation, relevant files and errors.
Patch Review	File names, before/after diff, explanation, accept/reject and test action.
Test Results	Command, container status, duration, exit status, stdout, stderr and pass/fail.
Evidence Report	Issue summary, findings, changed files, patch, tests, PR link and execution history.

The agent activity and patch review screens are especially important for judging. They must show real actions and results, not decorative progress animations.

12. Product Design Principles

- AI-first, but human-controlled.
- Evidence over unsupported AI claims.
- Simple user experience for beginners.
- Real integrations instead of simulated features.
- Modular engineering architecture.
- Security before autonomy.
- Functional depth over feature quantity.

13. Hackathon Development Priorities

Priority	Features
P0 — Critical	Foundation, GitHub integration, repository retrieval, issue analysis, file discovery, patch generation, Docker testing, evidence report, PR workflow and deployment.
P1 — Important	Agent timeline, duplicate issue detection, Repository Knowledge Map, screenshot analysis and beginner mode.
P2 — Future	Private repositories, more language runners, advanced semantic search, collaboration, health analytics and concurrent agent workflows.

14. Acceptance Criteria and Definition of Done

The MVP is complete when the following end-to-end workflow succeeds:

☐ A user connects a public GitHub repository.
☐ The application retrieves actual repository metadata and open issues.
☐ The user selects an issue.
☐ Gemma 4 analyzes the issue.
☐ RepoPilot retrieves relevant source files.
☐ The agent generates a proposed patch.
☐ The patch is validated and applied to a temporary workspace.
☐ Docker executes the configured tests.
☐ Actual test output and exit status are captured.
☐ A Patch Evidence Report is generated.
☐ The user reviews and approves the patch.
☐ RepoPilot creates a branch in the user's fork.

☐ RepoPilot opens a pull request against the original repository.
☐ The PR link appears in the application.

No simulated test results, fake GitHub operations or unverified success messages are acceptable.

15. Risks and Mitigation

Risk	Mitigation
Incorrect AI-generated code	Patch review, validation and testing.
Model API failure	Timeouts, safe retries and error reporting.
Large repository context	Bounded retrieval and file selection.
GitHub rate limits	Pagination, caching and request management.
Docker security risks	Isolated, restricted execution.
GitHub permission failure	Fork workflow and clear permission guidance.
Patch conflicts	Recheck base commit and reject stale patches.
Hackathon deadline	Prioritize P0 and one supported runtime.
Cloud execution limitations	Use a separate Docker worker if necessary.
Prompt injection	Treat repository content as untrusted and validate every tool call.

16. Final Product Definition

RepoPilot is a repository-aware AI engineering agent designed to simplify open-source contributions. It helps developers and maintainers transform reported software issues into investigated, tested and review-ready contributions.

Gemma 4 provides AI reasoning and code generation. GitHub provides source repositories and contribution workflows. React provides the developer experience. Node.js and Express coordinate the system. MongoDB stores application data. Docker provides isolated testing. DigitalOcean provides cloud infrastructure.

Defining Differentiator

The Patch Evidence Report: every proposed contribution should explain what changed, why it changed and what validation actually occurred.

Final Product Statement

RepoPilot helps developers and open-source maintainers transform reported software issues into investigated, tested and review-ready contributions while keeping humans in control of every published change.

Document note: This PRD defines product requirements and approved direction. Before implementation, verify current Gemma API capabilities, GitHub OAuth permissions, rate limits, DigitalOcean hosting constraints and Docker worker isolation options. Detailed schemas, sequence diagrams, request/response contracts and deployment configuration belong in the Technical Design Document (TDD).